

AURORA: A REPLICATION FOR THE COMP6258 RE-PRODUCIBILITY CHALLENGE

Chengyang Xie, Evan Vennall & Xiaoyu Yan
 Department of Electronics and Computer Science
 University of Southampton
 Hampshire, United Kingdom
 {cx1y25, ev4g22, xy1r25}@soton.ac.uk

ABSTRACT

This report presents a constrained reproduction of AuroRA. We implemented LoRA and AuroRA from scratch using PyTorch and evaluated them on three GLUE tasks using RoBERTa-base. Our results provide mixed support for AuroRA. AuroRA rank-2 substantially outperformed LoRA rank-2 on the CoLA and RTE datasets and matched LoRA rank-2 on MRPC. However, AuroRA rank-2 did not consistently match the performance of LoRA rank-8, especially with respect to MRPC and RTE. The ablation result was mixed. Removing the fixed tanh branch caused a large performance drop, while removing the spline branch had little effect. Overall, our reproduction suggests that AuroRA may effectively improve low-rank adaptation performance in some cases, but the effect was task-dependent under our constrained setup.

1 INTRODUCTION

Large pre-trained language models are commonly adapted for downstream tasks through fine-tuning. However, full fine-tuning updates all model parameters, which is computationally expensive. Parameter-Efficient Fine-Tuning (PEFT) methods reduce this cost by freezing the majority of pre-trained parameters and training only a small number of additional parameters, often around 2% of the total parameters of a model for LoRA and QLoRA ((Xu et al., 2023)).

LoRA is a widely used parameter-efficient fine-tuning method. It approximates the weight update during fine-tuning with two low-rank matrices. This reduces the number of trainable parameters, but it also introduces a low-rank bottleneck. When the rank is small, LoRA may not have enough expressive power to sufficiently approximate large models. Increasing the rank may improve performance, but it also increases the number of trainable parameters and thus training cost.

AuroRA attempts to address this issue by adding a nonlinear mapping between the two low-rank matrices. Instead of using only a linear low-rank update, AuroRA inserts an adaptive nonlinear layer. This is intended to improve expressiveness by learning non-linear relationships in the data, while keeping the rank small and training fast.

2 REPRODUCTION SCOPE

We did not use the official implementation. We implemented LoRA and AuroRA from scratch in PyTorch. Following the paper’s natural language understanding (NLU) setup, adapters were injected into the query and value projection layers of RoBERTa, while keeping the classification head fully trainable. All other RoBERTa parameters were frozen.

Our reproduction was constrained by computational resources. We used RoBERTa-base and three GLUE tasks ((Wang et al., 2019): MRPC, RTE, and CoLA. We did not reproduce the full set of experiments from the original paper, as the NLP and computer vision suites were trained and tested using four NVIDIA GeForce RTX 4090 (24GB) GPUs, and the commonsense reasoning and subject-driven generation suites with an NVIDIA L20 (48GB) (Dong et al. (2025)). Attempting to

replicate every result would have taken a longer time than we had available given the processing power used (one singular L4 GPU, with 24GB VRAM).

3 TARGET QUESTIONS

- a. Does AuroRA rank-2 outperform LoRA rank-2 under matched settings?
- b. Can AuroRA rank-2 approach the performance of higher-rank LoRA?
- c. Do the fixed tanh and learnable spline components both contribute to performance?

4 METHOD

LoRA:

$$\Delta h = B(Ax)$$

((Hu et al., 2021))

AuroRA:

$$\Delta h = B(\sigma(Ax))$$

where σ is implemented as:

$$\sigma(z) = \tanh(H(\tanh(z))) + \text{spline}(z)$$

((Dong et al., 2025))

The spline component was implemented using a self-written cubic B-spline basis. This differs from the official implementation, but preserves the central idea of a learnable nonlinear component.

In our implementation, LoRA and AuroRA were implemented as wrappers around existing linear layers. The original linear layer was frozen, and the adapter output was added to the original output. For LoRA, the adapter update was produced by two low-rank matrices. With AuroRA, the adaptive nonlinear layer (ANL) was inserted between these two matrices.

5 EXPERIMENTAL SETUP

Item	Setting
Model	roberta-base
Datasets	MRPC, RTE, CoLA
Methods	LoRA r=2, LoRA r=8, AuroRA r=2
Metrics	Accuracy for MRPC and RTE, MCC for CoLA
Seeds	0, 1, 2
Hardware	Google Colab L4 GPU, 24GB VRAM
Code	Self-implemented, no official AuroRA code

Table 1: Experimental setup.

The main experiments compare LoRA rank-2, LoRA rank-8, and AuroRA rank-2. LoRA rank-2 is used as the low-rank baseline. LoRA rank-8 is used as the higher-rank baseline. AuroRA rank-2 is used to test whether the nonlinear mapping improves low-rank adaptation.

6 MAIN RESULTS

The results provide partial support for AuroRA. AuroRA rank-2 substantially outperformed LoRA rank-2 in CoLA and RTE, and matched LoRA rank-2 in MRPC.

In MRPC, AuroRA rank-2 achieved a score of 0.868, the same as LoRA rank-2, with a lower variance of results. LoRA rank-8 achieved the a greater mean result, with 0.883, meaning that AuroRA rank-2 did not improve over LoRA rank-8 in MRPC.

Dataset	Method	Rank	Mean Score	SD
MRPC	LoRA	2	0.868	0.013
	LoRA	8	0.883	0.007
	AuroRA	2	0.868	0.004
RTE	LoRA	2	0.527	0.094
	LoRA	8	0.751	0.014
	AuroRA	2	0.681	0.037
CoLA	LoRA	2	0.000	0.000
	LoRA	8	0.000	0.000
	AuroRA	2	0.596	0.009

Table 2: Main results. Accuracy is used for MRPC and RTE. MCC is used for CoLA.

With RTE, AuroRA rank-2 achieved a score of 0.681, compared to 0.527 for LoRA rank-2. This shows a clear and significant improvement over the low-rank LoRA baseline. However, it still performed worse than LoRA rank-8, achieving 0.751.

Finally, in CoLA, both LoRA rank-2 and rank-8 collapsed to an MCC of 0.000 under our constrained setup, while AuroRA rank-2 achieved an MCC of 0.596. This is the strongest evidence in our reproduction that AuroRA can improve low-rank adaptation. However, because this collapse may be sensitive to hyperparameters and training stability, we avoid over-generalising this result.

6.1 TRAINABLE PARAMETERS

Method	Trainable parameters
LoRA r=2	665,858
AuroRA r=2	666,338
LoRA r=8	887,042

Table 3: Trainable parameter counts.

AuroRA rank 2 introduced only 480 more trainable parameters than LoRA rank 2, while using approximately 75.1% of the trainable parameters of LoRA rank 8.

7 ABLATION

The ablation compares full AuroRA, AuroRA without the fixed tanh component, and AuroRA without the spline component. This tests the degree to which the two nonlinear branches contribute individually and in complement to one another regarding low-rank adaptation improvements.

Setting	MRPC accuracy
Full AuroRA	0.868
AuroRA w/o fixed tanh	0.769
AuroRA w/o spline	0.866

Table 4: Ablation results on MRPC.

This ablation result is mixed. Removing the fixed tanh branch reduced MRPC accuracy from 0.868 to 0.769, indicating that the fixed nonlinear transformation was important in our implementation. However, whilst removing the spline branch did reduce accuracy, the reduction is minor, from 0.868 to 0.866. This differs from the original paper, where both components contributed to performance and the learnable spline component was reported to be more important. A likely reason is that our spline implementation was self-written and may not fully match the official implementation. Also

noteworthy is that the ablation was also only tested on MRPC, which limits the strength of our conclusion.

8 REFLECTION

Our constrained reproduction provides partial support for AuroRA. Whilst AuroRA rank-2 substantially outperformed LoRA rank-2 in CoLA and RTE, and matched LoRA rank 2 on MRPC, it could not consistently match up to a rank-8 LoRA, differing from the original paper’s findings that the expressive power gained from the ANL of AuroRA is greater even with a significantly lower rank than LoRA.

It is essential to note that the aspect of greatest difference between our reproduction and the original findings is the effectiveness of the non-linear B-splines. Whilst our fixed non-linear layer significantly improved on the performance relative to a simple linear layer as found in LoRA, our adaptive layer (which contains the B-splines) added negligible performance to the standard linear layer, as noted in the ablation. The original paper found that this adaptive layer contributed an even greater improvement than the fixed layer, so further work on our B-spline implementation may reveal greater results and closer match the original paper.

We believe that were the adaptive non-linear layer’s performance to fully match up with the original paper, this adaptation would be significant to future fine-tuning of ever-larger LLMs. The potential of a low-rank non-linear PEFT method being able to significantly outperform higher-rank linear PEFT adaptations, whilst increasing the number of trainable parameters compared to a similar-rank linear PEFT by a miniscule amount, would allow greater time-savings whilst simultaneously achieving higher performance. AuroRA would then perhaps become a standard approach for fine-tuning, due to these advantages, and also due to the conceptual approach being only slightly more complex than that of standard LoRA.

9 CONCLUSION

In this constrained reproduction, we found partial support for the claims presented in the original AuroRA paper. Whilst AuroRA rank-2 successfully outperformed the LoRA rank-2 baseline on the CoLA and RTE datasets, it failed to consistently match the performance of the higher-rank LoRA model. Furthermore, our ablation study revealed that the fixed non-linear transformation drove most of AuroRA’s performance gains, with our self-implemented B-spline component contributing negligibly—a notable divergence from the original findings. Despite these limitations, our reproductions reinforce the notion that introducing non-linear layers can effectively enhance low-rank adaptation without significantly increasing parameter counts. If future work on our B-spline implementation can align its performance with the original paper, AuroRA holds strong potential to become a highly efficient, standard approach for fine-tuning large language models.

REFERENCES

- Haonan Dong, Wenhao Zhu, Guojie Song, and Liang Wang. Aurora: Breaking low-rank bottleneck of lora with nonlinear mapping, 2025. URL <https://arxiv.org/abs/2505.18738>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019. URL <https://arxiv.org/abs/1804.07461>.
- Lingling Xu, Haoran Xie, Si-Zhao Joe Qin, Xiaohui Tao, and Fu Lee Wang. Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment, 2023. URL <https://arxiv.org/abs/2312.12148>.